

mistralai/**Mistral-Medium-3.5-128B** 

295

Follow

Mistral AI_

17.3k

Safetensors 24 languages mistral3 vLLM Eval Results fp8 License: other

⋮

Model card

Files

xet

Community 22

Mistral Medium 3.5 128B

Mistral Medium 3.5 is our first flagship merged model. It is a dense 128B model with a 256k context window, handling instruction-following, reasoning, and coding in a single set of weights. Mistral Medium 3.5 replaces its predecessor Mistral Medium 3.1 and Magistral in Le Chat. It also replaces Devstral 2 in our coding agent Vibe. Concretely, expect better performance for instruct, reasoning and coding tasks in a new unified model in comparison with our previous released models.

Reasoning effort is configurable per request, so the same model can answer a quick chat reply or work through a complex agentic run. We trained the vision encoder from scratch to handle variable image sizes and aspect ratios.

Find more information on our [blog](#).

To speed up local inference using vLLM or SGLang, check out our released [EAGLE model](#).

Downloads last month
18,272



Safetensors

Model size 128B params

Tensor type BF16 · F8_E4M3

Chat template

Files info

Inference Providers NEW

This model isn't deployed by any Inference Provider.



21

[Ask for provider support](#)

Model tree for mistralai/Mistral...

Finetunes 6 models

Quantizations 15 models

Spaces using mistralai/Mistra... 2

ayushKishor/plutoV2_miniProject_3r...

RadicalNotionAI/modelIdna

Collection including mistralai/Mi...

The Transformers config originally had an incorrect entry that caused long-context performance degradation. This has been fixed in this [commit](#). GGUFs generated using the Transformers config prior to this commit are also affected. Please use the correct config for best performance.

Key Features

Mistral Medium 3.5 includes the following architectural choices:

- **Dense 128B parameters.**
- **256k context length.**
- **Multimodal input:** Accepts both text and image input, with text output.
- **Instruct and Reasoning functionalities** with function calls (reasoning effort configurable per request).

Mistral Medium 3.5 offers the following capabilities:

- **Reasoning Mode:** Toggle between fast instant reply mode and reasoning mode, boosting performance with test-time compute when requested.
- **Vision:** Analyzes images and provides insights based on visual content, in addition to text.
- **Multilingual:** Supports dozens of languages, including English, French, Spanish, German, Italian, Portuguese, Dutch, Chinese, Japanese, Korean, and Arabic.
- **System Prompt:** Strong adherence and support for system prompts.

Mistral Medium 3.5  Collection

Our first flashi... • 2 items • U.. •  16

Evaluation results

Swe Bench Reso...  [leaderboard](#)  77.6

- **Agentic:** Best-in-class agentic capabilities with native function calling and JSON output.
- **Large Context Window:** Supports a 256k context window.

We release this model under a Modified MIT License: Open-source license for both commercial and non-commercial use with exceptions for companies with large revenue.

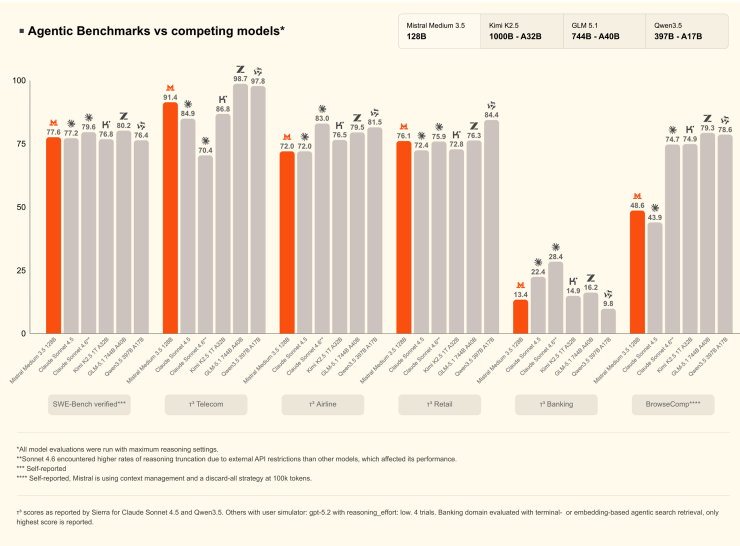
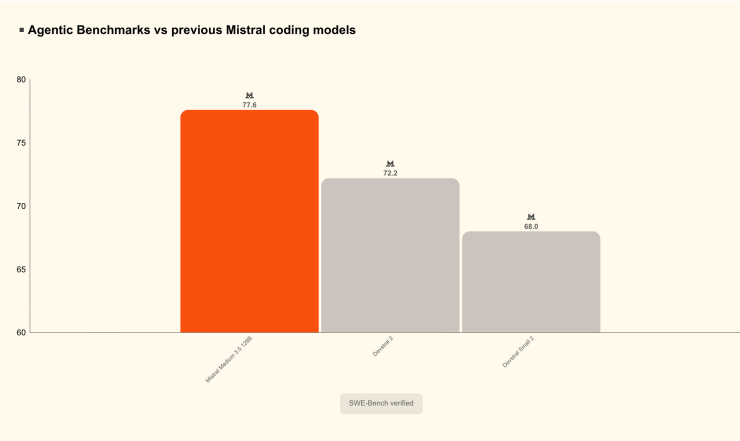
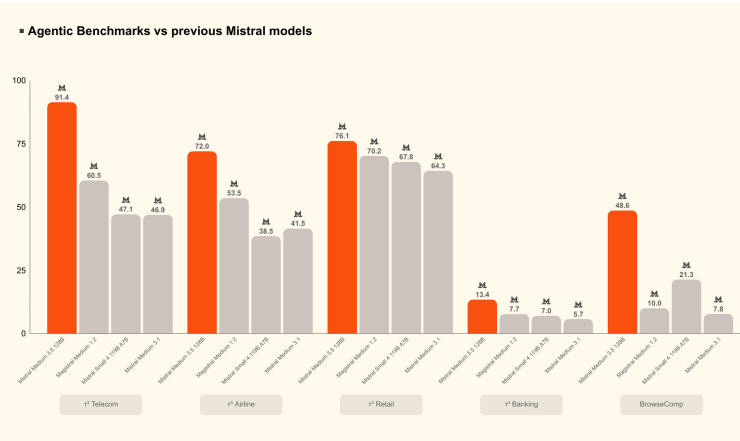
Recommended Settings

- **Reasoning Effort:**
 - 'none' → Do not use reasoning
 - 'high' → Use reasoning (recommended for complex prompts and agentic usage)
Use `reasoning_effort="high"` for complex tasks and agentic coding.
- **Temperature:** 0.7 for `reasoning_effort="high"`. Temp between 0.0 and 0.7 for `reasoning_effort="none"` depending on the task. Generally, lower means answer that are more to the point and higher allows the model to be more creative. It is a good practice to try different values in order to improve the model performance to meet your demands.
- **Top p:** 0.95 for `reasoning_effort="high"`. You can try different values but staying close should achieve best performance. Leave it to None (or 1.0) for `reasoning_effort="none"`.

Benchmarks

Agentic Benchmarks

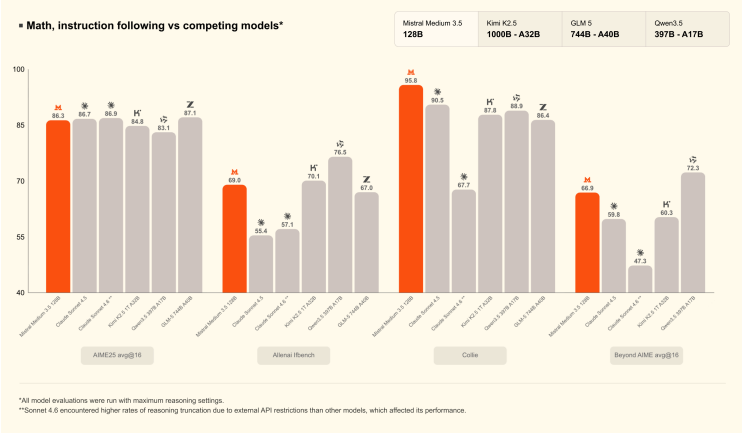
Mistral Medium 3.5 supersedes all our previous coding models, namely Devstral, across all benchmarks. It scores **91.4%** on τ^3 -Telecom and **77.6%** on SWE-Bench Verified. Due to its stronger agentic capabilities, Mistral Medium 3.5 replaces Devstral 2 in our coding agent, Vibe CLI.



Instruction Following, Reasoning, and Coding

Benchmarks

We compared Mistral Medium 3.5 with competing models on instruction following, reasoning (math), and coding benchmarks. Thanks to its unified capabilities, it achieves strong results across all these tasks and Mistral Medium 3.5 is now powering Le Chat.



Usage

You can find Mistral Medium 3.5 support on multiple libraries for inference and fine-tuning.

We here **thank** every contributors and maintainers that helped us making it happen.

Mistral-Vibe

Use Mistral Medium 3.5 with Mistral Vibe.

Install

Install the latest version:

```
uv pip install mistral-vibe --upgrade
```

API Usage

Mistral Medium 3.5 can be selected by starting `vibe`. If it is the first time you launch `vibe`, it will:

- Create a default configuration file at `~/vibe/config.toml`.
- Prompt you to enter your API key if it's not already configured.
- Save your API key to `~/vibe/.env` for future use.

Now select `mistral-medium-3.5` and start building !

Local server

If instead of pinging the Mistral API, you want to use a local vLLM server, you can do the following:

- 1. Spin up a vllm server as explained in [Usage - vllm](#)
- 2. Add the model configuration in `~/vibe/config.toml`:

```
display_name = "Mistral Medium 3.5 (local vLLM)"
description = "Mistral Medium 3.5 mode using vllm"
safety = "neutral"
```

```
active_model = "mistral-medium-3.5" # Make sure to use the correct model name
[[providers]]
name = "vllm"
api_base = "http://<your-host-url>:8000/v1"
api_key_env_var = ""
backend = "generic"
api_style = "reasoning"
```

```
[[models]]
name = "mistralai/Mistral-Medium-3.5-128B"
provider = "vllm"
```

```
alias = "mistral-medium-3.5"
thinking = "high"
temperature = 0.7
auto_compact_threshold = 168000

[tools.bash]
default_timeout = 1200
```

Notes:

- Make sure to overwrite `<your-host-url>` with your server's url.
- Other inference backends are also supported. Please look at [Mistral Vibe repo](#) for more info.

Then restart vibe and "tab-shift" to "mistral-medium-3.5" mode.

Give it a try on some coding agentic tasks and start building some cool stuff !

Inference

The model can be deployed with:

- [vllm](#) ([recommended](#)): See [here](#).
- [llama.cpp](#): See [here](#) for Unsloth's GGUFs.
- [LM studio](#): WIP stay tuned !
- [Ollama](#): See [here](#).
- [SGLang](#): See [here](#).
- [transformers](#): See [here](#).

For optimal performance, we recommend using the Mistral AI API if local serving is subpar.

Make sure that frameworks relying on the Transformers configuration, including GGUF files, are up to date with the fixes introduced in this [commit](#). Otherwise, you will experience subpar performance, especially in long-context sessions.

Fine-Tuning

Fine-tune the model via:

- [Axolotl](#): See [here](#).
- [Unsloth](#): See [here](#).

vLLM (Recommended)

We recommend using Mistral Medium 3.5 with the [vLLM library](#) for production-ready inference.

To speed up local inference using vLLM, check out our released [EAGLE model](#)

Installation

Make sure to install **vllm nightly**:

```
uv pip install -U vllm \
    --torch-backend=auto \
    --extra-index-url https://wheels.vllm.ai/
```

Doing so should automatically install [mistral_common >= 1.11.1](#) and [transformers >= 5.4.0](#).

To check:

```
python -c "import mistral_common; print(mistral_common.__version__)"
python -c "import transformers; print(transformers.__version__)"
```

You can also make use of a ready-to-go [docker image](#) or on the [docker hub](#).

Serve the Model

We recommend a server/client setup:

```
vllm serve mistralai/Mistral-Medium-3.5-128B
--tool-call-parser mistral --enable-auto-tool
--gpu_memory_utilization 0.8
```

Ping the Server

- ▶ Instruction Following
- ▶ Tool Call
- ▶ Vision Reasoning

SGLang

Serve Mistral Medium 3.5 with the [SGLang library](#) for production-ready inference.

To speed up local inference using SGLang, check out our released [EAGLE model](#).

Installation

Day-zero support ships in dedicated docker tags:

```
docker pull lmsysorg/sglang:dev-mistral-medium
docker pull lmsysorg/sglang:dev-cu13-mistral-medium
```

Or follow the [SGLang installation guide](#). Requires
transformers >= 5.4.0.

Serve the Model

```
python -m sglang.launch_server --model-path i
--tp 8 --tool-call-parser mistral --reason:
```

For the full deployment guide, benchmarks, and
per-request examples (reasoning effort, tool calls,
vision, streaming), see the [SGLang cookbook entry](#)
[for Mistral Medium 3.5](#).

Transformers

Installation

First install the [Transformers framework](#) to use
Mistral Medium 3.5:

```
uv pip install transformers
```

Inference

► Python Inference Snippet

License

This model is licensed under a [Modified MIT License](#).

*You must not use this model in a manner that
infringes, misappropriates, or otherwise violates any
third party’s rights, including intellectual property
rights.*